

A Comparative Analysis of Retrieval-Augmented Generation Architectures with Semantic Hashing for Enterprise Knowledge Systems

Hatice Altınok

R&D Center

Loodos Technology Inc.

Istanbul, Turkey

hatice.altinok@loodos.com

Hilal Tekgoz

R&D Center

Loodos Technology Inc.

Istanbul, Turkey

hilal.tekgoz@loodos.com

Halit Erdoğan

R&D Center

Loodos Technology Inc.

Istanbul, Turkey

halit.erdogan@loodos.com

Harun Uz

R&D Center

Loodos Technology Inc.

Istanbul, Turkey

harun.uz@loodos.com

Abstract—Large Language Models (LLMs) have significant potential to transform enterprise knowledge management. The Retrieval-Augmented Generation (RAG) architecture has become the industry standard for producing accurate, domain-specific responses while mitigating hallucinations. However, the retrieval stage in standard RAG implementations presents challenges in terms of latency, computational cost, and accuracy when scaling to large document collections. This paper presents a systematic comparative analysis of four distinct RAG retrieval architectures evaluated on a real-world enterprise wiki dataset containing 279 documents across 14 categories. The architectures examined include: (1) Dense Retrieval using FAISS, (2) Hybrid Retrieval combining FAISS with BM25, (3) Semantic Hashing-only retrieval, and (4) a novel two-stage hybrid model employing Semantic Hashing for candidate filtering followed by vector-based reranking. We evaluate these architectures using 84 domain-specific questions, measuring answer accuracy through semantic similarity, query latency, and memory consumption. Our experimental results demonstrate that the two-stage hybrid architecture achieves the highest accuracy rate of 82.14% with a mean response time of 2.96 seconds, outperforming both pure dense retrieval (78.57%) and hybrid BM25 approaches (64.29%). We provide detailed analysis of prompt engineering strategies that contributed to these results and discuss the trade-offs between retrieval speed, accuracy, and resource utilization for enterprise-scale RAG deployments.

Keywords—Retrieval-Augmented Generation, Semantic Hashing, Vector Search, Information Retrieval, Large Language Models, FAISS, BM25, Enterprise Knowledge Systems

I. INTRODUCTION

Organizations generate vast amounts of unstructured data, including wiki pages, project documentation, technical reports, and policy documents. Efficient access to this knowledge repository is critical for operational efficiency and decision-making. While Large Language Models (LLMs) offer powerful natural language understanding capabilities, their lack of domain-specific and up-to-date knowledge leads to “hallucinations”—plausible but factually incorrect outputs [1].

Retrieval-Augmented Generation (RAG) [1] has emerged as the dominant paradigm for addressing this limitation by connecting LLMs to external knowledge sources. The RAG architecture augments LLM responses with retrieved con-

text, improving factual accuracy and enabling domain-specific applications. However, as document collections scale, the retrieval component faces significant challenges in balancing accuracy, latency, and computational resources [7].

Recent surveys [5], [6] have categorized RAG systems based on their retrieval mechanisms, generation strategies, and augmentation techniques. Dense retrieval methods using approximate nearest neighbor (ANN) search [2] provide semantic matching capabilities but incur computational overhead that grows with corpus size. Sparse retrieval methods like BM25 [3] offer efficient keyword matching but may miss semantically related content. Semantic hashing techniques [4] provide an alternative approach by mapping documents to compact binary codes, enabling sub-linear search complexity.

This paper addresses the following research questions:

- 1) How do different retrieval architectures compare in terms of accuracy, latency, and memory usage for enterprise knowledge retrieval?
- 2) Can a two-stage hybrid approach combining semantic hashing with vector reranking outperform single-stage methods?
- 3) What role does prompt engineering play in maximizing RAG system performance?

Our contributions include: (1) a systematic evaluation of four RAG architectures on a real-world Turkish enterprise dataset, (2) a novel two-stage retrieval approach combining semantic hashing filtering with dense reranking, (3) detailed analysis of prompt engineering strategies for enterprise RAG applications, and (4) empirical insights into the trade-offs between retrieval efficiency and accuracy.

II. RELATED WORK

A. Retrieval-Augmented Generation

The RAG paradigm was introduced by Lewis et al. [1], who demonstrated that combining parametric (model weights) and non-parametric (retrieved documents) knowledge significantly improves performance on knowledge-intensive NLP tasks. The architecture consists of three stages: (1) indexing documents

into a searchable format, (2) retrieving relevant passages given a query, and (3) generating responses conditioned on retrieved context.

Recent advances in RAG can be categorized along several dimensions. Adaptive retrieval approaches, such as Self-RAG [8], enable models to dynamically decide when retrieval is beneficial, reducing unnecessary retrieval calls. Hierarchical retrieval methods like RAPTOR [9] recursively cluster and summarize documents to capture different levels of abstraction. Graph-based approaches such as GraphRAG [10] leverage knowledge graph structures to improve multi-hop reasoning. Corrective retrieval frameworks like CRAG [11] assess retrieved document quality and employ web search as a fallback mechanism.

For multilingual and low-resource languages, RAG systems face additional challenges. Chirkova et al. [12] examined RAG performance across languages and found that retrieval quality significantly impacts generation in multilingual settings. Recent work on Turkish NLP, including Turk-LettuceDetect [13], has begun addressing hallucination detection for Turkish RAG applications, highlighting the morphological complexity of Turkish as a challenge for both retrieval and generation.

B. Dense and Sparse Retrieval

Dense retrieval methods transform queries and documents into continuous vector representations using neural encoders. The Dense Passage Retriever (DPR) [14] demonstrated that bi-encoder architectures trained on question-passage pairs outperform BM25 on open-domain QA. FAISS [2] provides efficient ANN search using techniques such as inverted file indexing (IVF) and product quantization (PQ), enabling billion-scale similarity search.

Sparse retrieval methods, exemplified by BM25 [3], compute relevance scores based on term frequency and inverse document frequency. Despite being “bag-of-words” approaches, they remain competitive baselines and excel at exact keyword matching [15].

Hybrid approaches combine dense and sparse methods to leverage their complementary strengths. The Ensemble Retriever pattern [16] merges results from multiple retrievers using reciprocal rank fusion or learned weighting. Wang et al. [17] conducted extensive experiments on hybrid RAG configurations, finding that the optimal combination depends on domain characteristics.

C. Semantic Hashing and Locality-Sensitive Hashing

Semantic hashing [4] maps high-dimensional embeddings to compact binary codes while preserving semantic similarity. Documents are encoded such that semantically similar documents hash to nearby addresses, enabling retrieval by simply accessing addresses within a small Hamming distance from the query hash. This approach is faster than locality-sensitive hashing (LSH) for approximate matching [4].

LSH [18], [19] is a data-independent hashing technique that ensures similar items hash to the same bucket with

high probability. Recent work has extended LSH to structured data [20] and developed neural approaches for learning hash functions [21]. The DET-LSH algorithm [22] achieves accuracy comparable to state-of-the-art ANN methods while significantly reducing query time.

For information retrieval applications, semantic hashing offers several advantages: (1) constant-time lookup via Hamming distance computation, (2) compact memory footprint using binary codes, and (3) the ability to quickly filter candidates before applying more expensive similarity measures. He et al. [23] proposed a robust semantic hashing framework for similar text search that addresses adversarial perturbations.

III. METHODOLOGY

A. System Architecture Overview

We evaluate four retrieval architectures, illustrated in Figure 1. All architectures share common preprocessing and generation components but differ in their retrieval mechanisms.

Model A: Dense Only

Query → FAISS → LLM

Model B: Hybrid

Query → FAISS + BM25 → Fusion → LLM

Model C: SH Only

Query → SH → LLM

Model D: Two-Stage Hybrid

Query → SH Filter → VS Rerank → LLM

Fig. 1. Overview of four RAG retrieval architectures evaluated in this study. SH = Semantic Hashing, VS = Vector Search.

B. Document Processing Pipeline

Documents are processed through a common pipeline:

Loading: We support PDF, Markdown (.md), and plain text formats using PyPDFLoader, UnstructuredMarkdownLoader, and TextLoader from the LangChain framework.

Chunking: Documents are split using RecursiveCharacterTextSplitter with `chunk_size=1500` characters and `chunk_overlap=250` characters. This configuration balances context preservation with retrieval granularity.

Embedding: Text chunks are encoded using Google’s embedding-001 model, producing 768-dimensional vectors. The same embedding model is used for both document indexing and query encoding.

C. Model A: Dense Retrieval (FAISS)

Model A implements standard dense retrieval using FAISS [2]. Given a query q , we compute its embedding $e_q \in \mathbb{R}^{768}$ and retrieve the k nearest neighbors from the document embedding index using cosine similarity:

$$\text{sim}(e_q, e_d) = \frac{e_q \cdot e_d}{\|e_q\| \|e_d\|} \quad (1)$$

We use FAISS’s IndexFlatIP (inner product) index with L2-normalized vectors, which is equivalent to cosine similarity. We set $k = 3$ to retrieve the top-3 most similar chunks.

D. Model B: Hybrid Retrieval (FAISS + BM25)

Model B combines dense and sparse retrieval using LangChain's EnsembleRetriever. BM25 scores are computed as:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})} \quad (2)$$

where $f(t, d)$ is the term frequency, $|d|$ is document length, avgdl is average document length, and $k_1 = 1.2$, $b = 0.75$ are BM25 parameters.

Results from both retrievers are merged with equal weights (0.5 each) using reciprocal rank fusion, retrieving $k = 5$ documents from each retriever before fusion.

E. Model C: Semantic Hashing Only

Model C implements semantic hashing based on random projection [4]. We project 768-dimensional embeddings to 128-bit binary codes using a fixed random matrix $P \in \mathbb{R}^{768 \times 128}$:

$$h(e) = \text{sign}(e \cdot P) = \begin{cases} 1 & \text{if } (e \cdot P)_i \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

We selected 128 bits after evaluating 64, 128, 256, and 512-bit configurations. While 256-bit hashes improved Recall@5 marginally (0.66 vs 0.64), they doubled computation time. The 128-bit configuration provides optimal balance between retrieval quality and efficiency.

Retrieval is performed by computing the Hamming distance between query and document hashes:

$$d_H(h_q, h_d) = \sum_{i=1}^{128} \mathbf{1}[h_q^{(i)} \neq h_d^{(i)}] \quad (4)$$

Documents are ranked by Hamming distance (ascending), and the top- k results are returned. The projection matrix is initialized with a fixed random seed (42) for reproducibility.

F. Model D: Two-Stage Hybrid (SH + Vector Reranking)

Model D implements a two-stage retrieval approach that combines the efficiency of semantic hashing with the precision of dense retrieval:

Stage 1 (Filtering): Use semantic hashing to rapidly filter the corpus, retrieving a candidate set of $n = 100$ documents with lowest Hamming distances.

Stage 2 (Reranking): Apply vector similarity search on the candidate set to select the final top- $k = 5$ results.

This approach reduces the search space for the expensive vector comparison from N (full corpus) to n (candidates), providing computational efficiency while maintaining retrieval quality.

G. Generation Component

All models use Google's Gemini 1.5 Pro as the generation model with temperature=0.1 for deterministic outputs. The retrieved context is concatenated and passed to the model along with a carefully engineered prompt.

H. Prompt Engineering Strategy

A critical factor in Model D's performance is the detailed prompt template, which includes:

- 1) **Categorical response rules:** Predefined responses for common query categories (greetings, thanks, identity questions, company information) to ensure consistent handling of non-retrieval queries.
- 2) **Fallback behavior:** Explicit instruction to respond "Bu bilgi belgelerde bulunmamaktadır" (This information is not found in the documents) when relevant context is unavailable.
- 3) **Context grounding:** Clear separation of context and query sections with explicit instructions to base answers on provided context.
- 4) **Domain knowledge injection:** Static company information embedded in the prompt to handle frequently asked organizational questions without retrieval.

IV. EXPERIMENTAL SETUP

A. Dataset Description

We use a proprietary enterprise wiki dataset from a Turkish technology company. The dataset contains 279 total documents in PDF and Markdown formats, in Turkish language, spanning 14 categories with approximately 1,850 total chunks after splitting (average chunk length: 1,200 characters).

The documents span 14 categories: Human Resources (procedures, benefits, onboarding), IT Support (VPN, hardware guides), Project Management (Jira usage, Scrum rituals), Software Development (Android, iOS, Backend, Frontend, DevOps standards), Quality Assurance (test procedures, DoD/DoR), Business Analysis (analysis formats, project documentation), Design (design onboarding), and R&D (project documentation, life cycle).

B. Evaluation Dataset

We created a golden evaluation dataset of 84 question-answer pairs covering diverse topics from the corpus. Questions were manually crafted to represent realistic enterprise queries.

C. Evaluation Metrics

Answer Success Rate (%): Cosine similarity between generated and reference answer embeddings, with threshold ≥ 0.80 for success. The threshold was validated via expert review; manual verification on 20% sample showed 94% agreement with automatic scores.

Retrieval Metrics: Recall@ k and Mean Reciprocal Rank (MRR = $\frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$).

Efficiency Metrics: Mean response time (seconds) and memory usage (MB) via psutil.

D. Implementation Details

All experiments were conducted on a system with Python 3.10, LangChain 0.1.x framework, Google Generative AI (embedding-001, gemini-1.5-pro), FAISS (CPU version), and NumPy for hash operations.

V. RESULTS AND ANALYSIS

A. Overall Performance Comparison

Table I presents the comparative results across all four architectures, including 95% confidence intervals computed via bootstrap resampling (1000 iterations).

TABLE I
PERFORMANCE COMPARISON OF RAG ARCHITECTURES

Model	Success (%)	95% CI	R@5	MRR	Time (s)
A (FAISS)	78.57	[69.0, 86.9]	0.79	0.72	2.43
B (Hybrid)	64.29	[53.6, 73.8]	0.71	0.58	3.43
C (SH)	58.33	[47.6, 69.0]	0.64	0.51	1.83
D (SH+VS)	82.14	[73.8, 89.3]	0.83	0.76	2.96

B. Key Findings

Finding 1: Two-Stage Hybrid Achieves Highest Accuracy. Model D (SH+VS Rerank) achieves the highest success rate of 82.14%, outperforming pure dense retrieval (Model A) by 3.57 percentage points. This demonstrates that the two-stage approach effectively combines fast candidate filtering with precise reranking.

Finding 2: BM25 Hybridization Degrades Performance. Counter to expectations, Model B performs 14.28 percentage points worse than Model A. Additional analysis revealed: (1) Turkish morphological variations cause term-matching failures—testing with Zemberek stemming improved Model B to 71.4%; (2) Manual inspection of 20 failed cases showed 65% involved keyword ambiguity (e.g., “izin prosedürü” retrieving “erişim izni” documents due to shared “izin” term).

Finding 3: Semantic Hashing Alone Is Insufficient. Model C achieves the fastest response time (1.83s) but the lowest accuracy (58.33%). The lossy compression of 768-dimensional embeddings to 128-bit hashes discards fine-grained semantic information needed for accurate retrieval. However, semantic hashing remains effective as a first-stage filter.

Finding 4: Memory Efficiency of Hashing. Models using semantic hashing (C, D) consume less memory (~269-274 MB) compared to pure FAISS models (~289 MB). The binary hash representations are more compact than full embedding vectors.

Finding 5: Effect of k . Varying the number of retrieved documents (k) affects performance differently across models. Model A peaks at $k = 3$ (78.57%) and degrades to 70.24% at $k = 10$ as irrelevant documents dilute context. Model D shows more stable performance (80.95%–82.14% for $k \in [3, 10]$) due to quality filtering in Stage 1.

C. Error Analysis

Analysis of failure cases revealed: Model A failures primarily occurred when relevant context was not in top-3 chunks; Model B failures resulted from BM25 retrieving keyword-overlapping but semantically different documents; Model C failures stemmed from hash collisions causing retrieval of

semantically distant documents; Model D failures occurred when correct documents were not in the top-100 Stage 1 candidates.

D. Impact of Prompt Engineering

We conducted ablation experiments on Model D to assess prompt engineering impact. Removing categorical response rules decreased success rate by 8.3%, as the model attempted retrieval for greetings and identity queries, often returning irrelevant context. Removing the fallback instruction increased hallucination rate by 12% for out-of-scope queries.

VI. DISCUSSION

Our results highlight key design considerations for enterprise RAG: (1) No universal best architecture—optimal choice depends on corpus characteristics and resource constraints; (2) Careful hybrid design required—naive retriever combination can degrade performance; (3) Prompt engineering matters—well-designed prompts with explicit rules significantly impact end-to-end performance.

For larger corpora, we expect Model D’s advantage to increase as hash filtering provides greater relative speedup. The 279-document corpus is modest; efficiency benefits would be more pronounced with millions of documents where exhaustive vector search becomes prohibitive.

A. Limitations

Key limitations include: (1) Dataset size—84 evaluation questions provide limited statistical power; confidence intervals in Table I show overlapping ranges for some comparisons; (2) Single domain—results are specific to Turkish enterprise documentation; (3) Single LLM—only Gemini 1.5 Pro tested; (4) Evaluation metric—embedding similarity may miss nuanced quality differences; (5) Language-specific—BM25 findings may not generalize to morphologically simpler languages.

VII. CONCLUSION AND FUTURE WORK

This paper presented a systematic comparison of four RAG retrieval architectures for enterprise knowledge systems. Our experiments on a real-world Turkish enterprise wiki dataset demonstrated that a two-stage hybrid approach combining semantic hashing filtering with vector-based reranking achieves the highest accuracy (82.14%) while maintaining acceptable latency (2.96s). We showed that naive BM25 hybridization can degrade performance and that prompt engineering plays a critical role in end-to-end RAG effectiveness.

Future work will focus on: (1) validation on larger-scale datasets including WikiRAG-TR; (2) investigation of learned hash functions that preserve more semantic information; (3) integration of cross-encoder rerankers in the second stage; (4) optimization of memory efficiency by storing only document IDs in the hash index with on-demand content retrieval; and (5) extension to multilingual enterprise settings.

ACKNOWLEDGMENT

This research was conducted at the Loodos Technology R&D Center with support from the Turkish Ministry of Industry and Technology.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] R. Salakhutdinov and G. Hinton, "Semantic hashing," *Int. J. Approximate Reasoning*, vol. 50, no. 7, pp. 969–978, 2009.
- [5] S. Gupta, R. K. Ranjan, and K. Singh, "A comprehensive survey of retrieval-augmented generation (RAG): Evolution, current landscape and future directions," arXiv preprint arXiv:2410.12837, 2024.
- [6] P. Zhao, H. Zhang, Q. Yu, et al., "Retrieval-augmented generation for AI-generated content: A survey," arXiv preprint arXiv:2402.19473, 2024.
- [7] Y. Gao, Y. Xiong, A. Rajput, et al., "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2024.
- [8] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," arXiv preprint arXiv:2310.11511, 2023.
- [9] P. Sarthi, S. Abdullah, A. Tuli, et al., "RAPTOR: Recursive abstractive processing for tree-organized retrieval," arXiv preprint arXiv:2401.18059, 2024.
- [10] D. Edge, H. Trinh, N. Cheng, et al., "From local to global: A graph RAG approach to query-focused summarization," arXiv preprint arXiv:2404.16130, 2024.
- [11] S. Yan, C. Gu, Y. Guo, et al., "Corrective retrieval augmented generation," arXiv preprint arXiv:2401.15884, 2024.
- [12] N. Chirkova, D. Rau, H. Déjean, et al., "Retrieval-augmented generation in multilingual settings," arXiv preprint arXiv:2407.01463, 2024.
- [13] M. Yilmaz et al., "Turk-LettuceDetect: Hallucination detection models for Turkish RAG applications," in *Proc. Turkish NLP Workshop*, 2024.
- [14] V. Karpukhin, B. Oğuz, S. Min, et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, pp. 6769–6781, 2020.
- [15] J. Lin, R. Nogueira, and A. Yates, "Pretrained transformers for text ranking: BERT and beyond," *Synthesis Lectures on Human Language Technologies*, vol. 14, no. 4, pp. 1–325, 2021.
- [16] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting for retrieval-augmented large language models," arXiv preprint arXiv:2305.14283, 2023.
- [17] X. Wang, Q. Liu, Y. Gui, H. Lin, and W. Wu, "Searching for best practices in retrieval-augmented generation," arXiv preprint arXiv:2407.01219, 2024.
- [18] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions," in *Proc. SCG*, pp. 253–262, 2004.
- [19] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. STOC*, pp. 604–613, 1998.
- [20] W. Wu, B. Li, C. Luo, W. Nejdl, and X. Tan, "Hierarchical locality sensitive hashing for structured data: A survey," arXiv preprint arXiv:2204.11209, 2024.
- [21] R. Wang et al., "Neural locality sensitive hashing for entity blocking," arXiv preprint arXiv:2401.18064, 2024.
- [22] Y. Zhang et al., "DET-LSH: A locality-sensitive hashing scheme with dynamic encoding tree for approximate nearest neighbor search," *IEEE Trans. Knowledge and Data Engineering*, 2024.
- [23] L. He, Z. Huang, E. Chen, et al., "An efficient and robust semantic hashing framework for similar text search," *ACM Trans. Information Systems*, 2023.
- [24] Metin, "WikiRAG-TR: Turkish Wikipedia RAG evaluation dataset," Hugging Face Datasets, 2024. [Online]. Available: <https://huggingface.co/datasets/Metin/WikiRAG-TR>